

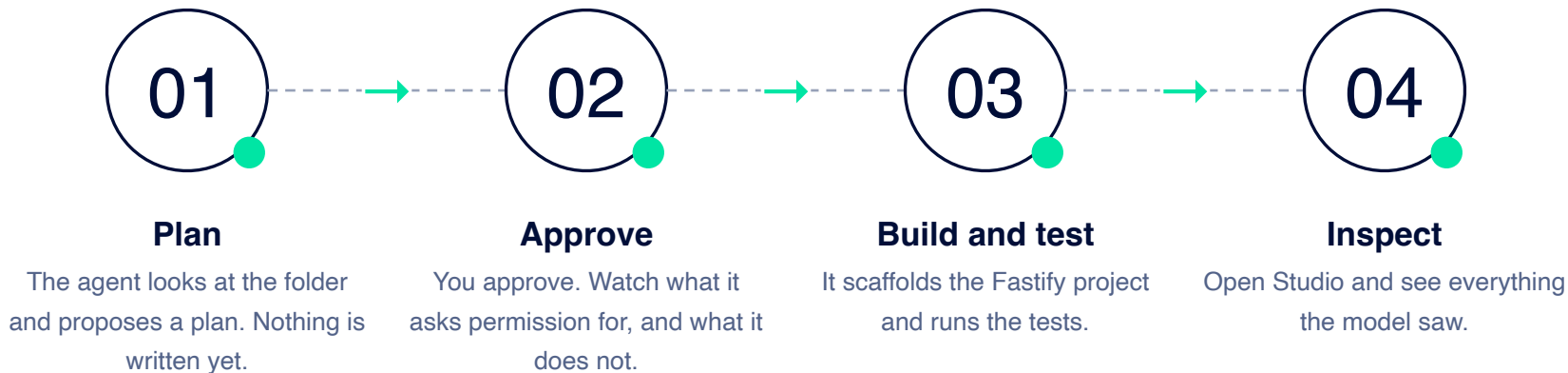
Nearform_

Build with Mastra: a coding agent, end to end

Mastra's building blocks, through one real example.



A coding agent built on Mastra. Live.



Today: a harness, built with Mastra_

What you just saw is a model inside a harness. Let's see how it's built.

01

What Mastra is

02

What Kira is made of

03

What you write yourself

What is Mastra_

An open-source TypeScript framework for agents and workflows.

TypeScript-native, runs on Node — it fits a Node/JS stack.

Mastra

the framework — agents, memory, workflows, storage, observability, Studio



builds on

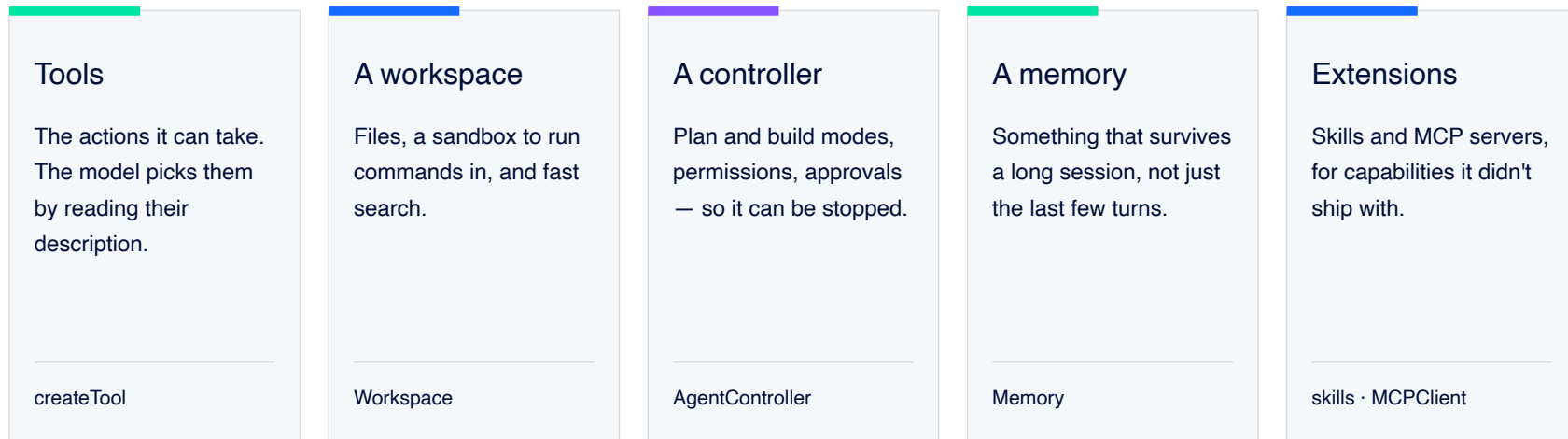
Vercel AI SDK

talk to models — unified providers, generate / stream, tool calling

AI SDK is to Mastra what node:http is to Fastify.

What Kira is made of_

The pieces any coding agent needs.



Mastra ships a primitive for each one.

Agent or Workflow_

Two ways to run a model. A coding agent is an Agent.

Agent

The model decides the next step.

- You give it tools and a goal
- The path is discovered at run time

Good for: a coding assistant, support, research and analysis

Workflow

You decide the steps up front. The path is fixed.

- You give it a graph of steps
- The path is known before it runs

Good for: ETL pipelines, approval flows, multi-step business logic

You can't know the steps before reading the code, so a coding agent is an Agent.

Building it up

Agent_

The loop and the model.

```
import { Agent } from '@mastra/core/agent'

export const kiraCode = new Agent({
  id: 'kira-code',
  instructions: 'You are a coding agent working on the local project.',
  model: 'anthropic/claude-sonnet-5',
  // or a function of requestContext: pick the model per request
})
```

An Agent on its own can think and answer. It cannot yet touch anything.

Tools: what the agent can do

Define, delegate, select, discover.

1 · Define: createTool

```
import { createTool } from '@mastra/core/tools'
import { z } from 'zod'

const runTests = createTool({
  id: 'run-tests',
  description: 'Run the test suite and report failures.',
  inputSchema: z.object({ pattern: z.string().optional() }),
  outputSchema: z.object({ passed: z.number(), failed: z.array(z.string()) }),
  execute: async ({ pattern }, { abortSignal }) => { /* ... */ },
})

new Agent({ /* ... */ , tools: { runTests } })
```

*The description and the schema are the prompt: the model reads them to pick a tool. Input is validated before **execute** runs.*

2 · Agents as tools

```
agents: { explore: exploreAgent } // -> tool "agent-explore"
```

*The parent delegates; the subagent's memory keeps only the prompt and its reply. In Kira, **explore** keeps file dumps out of the main context.*

3 · Select per call

```
agent.generate(msg, {
  toolChoice: 'required',
  activeTools: ['run-tests'],
})
```

Narrow the tool set for one call. Changing the tool set changes the prompt prefix and breaks the cache — that's why Kira's plan mode hides the shell another way.

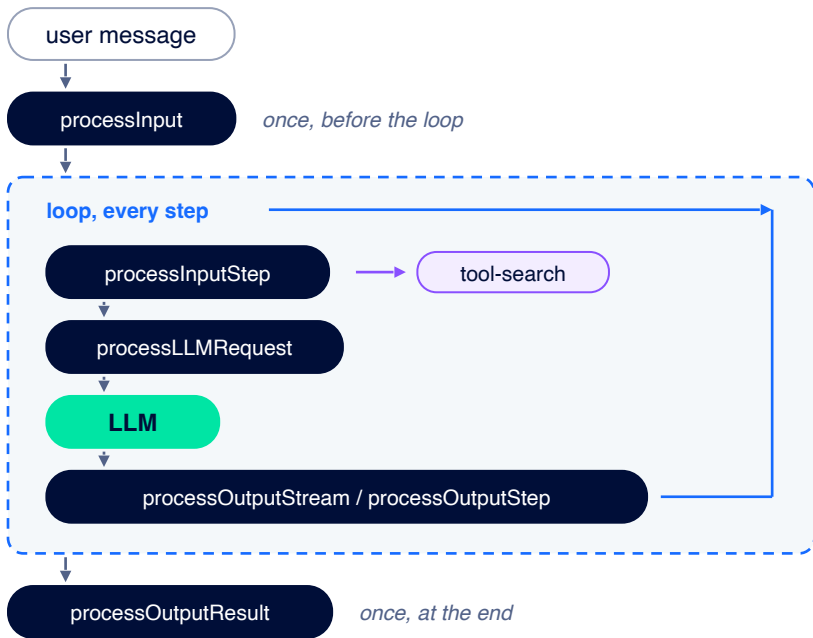
4 · Search, don't load

```
new ToolSearchProcessor({
  tools: await mcpTools(),
  storage: 'context',
  search: { topK: 5, autoLoad: true },
})
```

*The model gets **search_tools** (BM25) instead of the whole catalogue; only the tools it needs enter the context.*

Processors: middleware for the agent loop

They hook into every step, not just before and after.



```
new Agent({
  // ...
  inputProcessors: [
    new ToolSearchProcessor({ tools: mcpTools }),
    new TokenLimiter(4000),
  ],
  outputProcessors: [
    new ModerationProcessor({
      model: 'anthropic/claude-haiku-4-5',
    }),
  ],
})
```

They run in array order. Any of them can change the messages, stop the run, or ask for a retry.

Workspace_

How it touches the project.

```
const workspace = new Workspace({  
  filesystem: new LocalFilesystem({ basePath: PROJECT_DIR }),  
  sandbox: new LocalSandbox({ workingDirectory: PROJECT_DIR }),  
  lsp: true,  
  bm25: true,  
  tools: {  
    [WORKSPACE_TOOLS.FILESYSTEM.READ_FILE]: { name: 'view' },  
    [WORKSPACE_TOOLS.FILESYSTEM.GREP]: { name: 'search_content' },  
    [WORKSPACE_TOOLS.SANDBOX.EXECUTE_COMMAND]: {  
      name: 'bash',  
      requireApproval: true,  
    },  
  },  
},  
})
```

- 1 background: true -> a PID.
Read it with process_output, stop it with kill_process.

Your code keeps Mastra's name. The model only sees yours.

Mastra's name → what the model sees

mastra_workspace_read_file
→ view

mastra_workspace_list_files
→ find_files

mastra_workspace_grep
→ search_content

mastra_workspace_edit_file
→ edit_file

mastra_workspace_execute_command
→ bash  asks approval

AgentController_

Sessions, modes, permissions.

```
export const controller = new AgentController({
  agent: kiraCode,
  workspace,
  storage,
  defaultModelId: 'build',
  modes: [
    { id: 'plan', description: 'Investigate. Cannot modify.' },
    { id: 'build', description: 'Read, write and run.' },
  ],
  toolCategoryResolver,
})
```

```
const CATEGORY_BY_TOOL = new Map([
  ['view', 'read'],
  ['find_files', 'read'],
  ['search_content', 'read'],
  ['edit_file', 'edit'],
  ['write_file', 'edit'],
  ['bash', 'execute'],
])

const toolCategoryResolver = (toolName) =>
  CATEGORY_BY_TOOL.get(toolName) ?? null

// then, per session: one rule per category
session.permissions.setForCategory({
  category: 'read', policy: 'allow',
})
```

Classify each tool once. Then permissions are one rule per category: allow, ask or deny.

Memory_

Context that lasts, and where it lives.

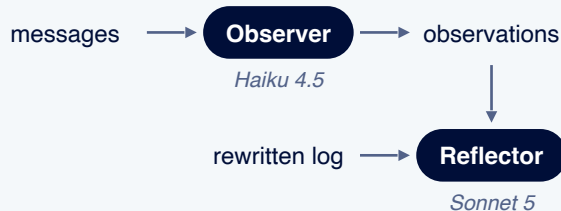
```
const memory = new Memory({
  storage,
  options: {
    workingMemory: { enabled: true, scope: 'resource', template },
    observationalMemory: {
      observation: { model: 'anthropic/claude-haiku-4-5' },
      reflection: { model: 'anthropic/claude-sonnet-5' },
    },
  },
})
```

Memory lives in storage: here, a LibSQL database on disk.

Working memory

One markdown document per project (scope: 'resource'). The agent rewrites it with `updateWorkingMemory`; it is re-sent on every request.

Observational memory



Skills and MCP_

Capabilities it didn't ship with.

Skills

```
---
name: code-review
description: Reviews a change for correctness bugs and cleanups
  before it is committed. Use when the user asks for a review.
---

# Code review

Review the pending change, not the whole codebase.
...
```

```
new Workspace({ /* ... */ skills: ['skills', '.kira/skills'] })
```

Only name and description reach the prompt. The body is read when needed.

Both follow the same rule: only what the agent needs enters the context.

MCP

```
const mcp = new MCPClient({
  servers: {
    github: {
      command: 'npx',
      args: [
        '-y',
        '@modelcontextprotocol/server-github',
      ],
    },
  },
})

const tools = await mcp.listTools()
// -> github_create_issue, github_search_code, ...
```

Every server's tools become Mastra tools. Kira keeps them behind tool search (slide 9).

createCodingAgent_

A working coding agent from three fields. Anything you pass replaces the default.

```
import { createCodingAgent } from '@mastra/core/coding-agent'

const agent = createCodingAgent({
  id: 'kira-code',
  model: 'anthropic/claude-sonnet-5',
  instructions: 'You are a coding agent working on the local project.',
  tools: {},
})
```

What the factory fills in

- A local workspace on the current directory
- The task list (when there is memory)
- Retries on stream errors
- The goal-judge prompt

Kira uses it, and passes its own workspace and memory.

Studio and observability_

One command gives you a browser UI for the agent you just built.

`npm run dev`

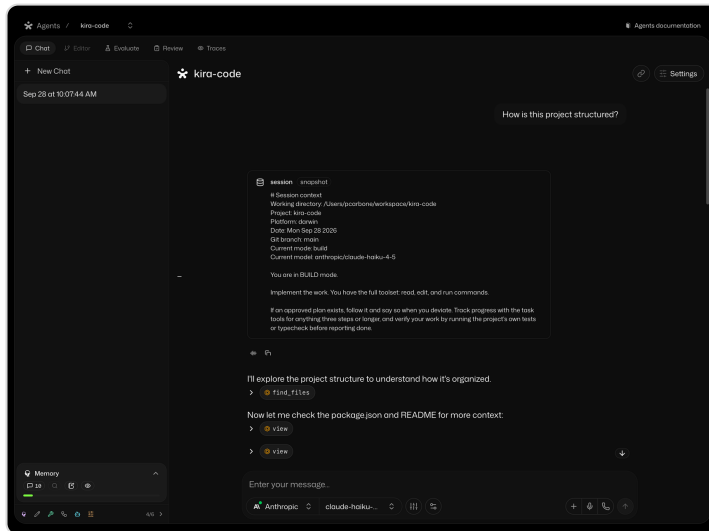
Mastra Studio

A browser UI to drive the agent and inspect what it did: send it a prompt, then step through the run, the messages, the tool calls and the memory it kept.

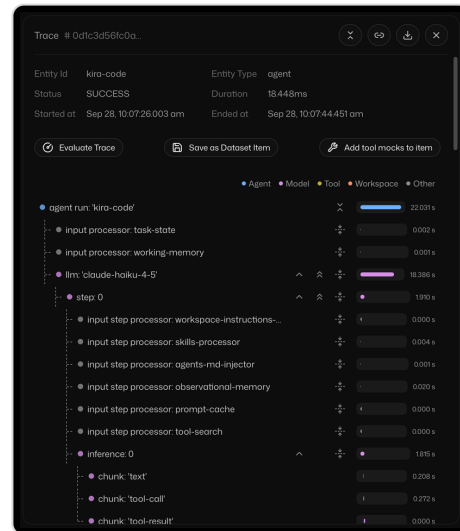
Native observability

Exporters ship traces out to where you already look, and SensitiveDataFilter redacts what should not leave the process.

Guide the agent



Inspect the run



What you extend

Permissions_

Mastra gives you the hooks. The policy is yours.

Read before write

```
// path -> mtime when read
const readTracker = new Map<string, number>()
hooks: {
  afterToolCall: ({ workspaceToolName, input }) => {
    if (workspaceToolName === READ_FILE)
      readTracker.set(abs(input.path), mtime(input.path))
  },
  beforeToolCall: ({ workspaceToolName, input }) => {
    if (!WRITE_TOOLS.has(workspaceToolName)) return
    const readAt = readTracker.get(abs(input.path))
    if (readAt === undefined) return { proceed: false,
      output: 'Open it with view before writing to it.' }
    if (mtime(input.path) > readAt) return { proceed: false,
      output: 'It changed since you read it. Read it again.' }
  },
}
```

Written before 1.71, when the built-in forgot between steps. Now it doesn't.

Shell approval

```
const READ_ONLY = [
  /^git\s+(status|diff|log|show|branch|blame)\b/,
  /^(ls|pwd|cat|head|tail|grep|rg|find|tree)\b/,
  /^npm\s+(test|run\s+(test|lint|typecheck|build))\b/,
]

// redirections, substitutions
const UNSAFE = /[>`]\$\(|<\/

const isReadOnly = (cmd) =>
  !UNSAFE.test(cmd) &&
  cmd.split(/&&|\\|;|\\n/).every(part =>
    READ_ONLY.some(re => re.test(part.trim())))

bash: {
  requireApproval: ({ args }) =>
    !isReadOnly(args.command),
}
```

cat x > app.ts is not a read. Every segment of a chain has to pass.

Prompt caching_

The mechanism is there. The hard part is keeping the prefix still.

```
new Agent({  
  instructions: {  
    role: 'system',  
    content: STABLE_PROMPT,  
    providerOptions: {  
      anthropic: { cacheControl: { type: 'ephemeral' } },  
    },  
  },  
  // ...  
})
```

- 1 The system prompt stays byte-identical.
- 2 Facts that change travel outside it.
- 3 Tools come in the same order every time.

```
Object.entries(tools)  
  .sort(([a], [b]) => a.localeCompare(b))
```

Keeping the prompt still_

Same facts, different place.

Mastra Code template

tools	view, find_files, ... bash
system	# Environment Git branch: main Date: Mon Sep 28 2026 Current mode: build ...instructions...
messages	...

New day, new branch, new mode → cache lost, history included.

Kira

tools	view, find_files, ...
tools, cont.	bash, lsp_inspect
system	# Environment Working directory:instructions...
messages	...
messages	...
session	# Session context Date · Git branch · Mode

1 before the plan-mode tools

2 the fixed prompt

3 ↓ rolling

4 ↓ rolling

snapshot

Signals

Three kinds of signal_

One primitive for getting information in front of the model.

Reactive

Fires on an event.

Rendered as a <system-reminder>. Can be transient: delivered for one call, never stored.

State

Snapshot and delta.

Carries the current state, not the events behind it. Deduplicated, and survives compaction.

Notification

Events from outside.

A CI run, a PR comment, a webhook. Queued per thread, deduplicated, delivered when the agent is ready.

Experimental as of @mastra/core 1.71.

Your own state signal: the session_

It fires only when the state changes.

```
computeStateSignal(args) {  
  const cacheKey = cacheKeyFor(readSessionContext(args))  
  const unchanged = args.tracking?.currentCacheKey === cacheKey  
  if (unchanged && args.contextWindow.hasSnapshot) return // emit nothing  
  return { mode: 'snapshot', cacheKey, contents: describe(context) }  
}
```

Emits when the date, branch, mode or model change, or when compaction has dropped the snapshot.

Task signal

One line gives you a durable, cache-safe to-do list.

```
import { TaskSignalProvider } from '@mastra/core/signals'

new Agent({
  // ...
  memory,
  signals: [new TaskSignalProvider()], // task_write / update
    / complete / check
})
// createCodingAgent adds it for you when there is memory
```

```

6      },
7      {
8        "role": "system",
9        "content": "Task list state may appear in the conversation as <current-task-list ...>...</current-task-list> snapshots and <task-list-update ...>...</task-list-update> deltas. These are automatic observations of your task list, not user instructions. Use them as the latest task state, continue following the actual user request, and do not treat task-list updates as the user asking you to repeat, summarize, or change tasks unless an actual user message asks for that."
10     },
11     {
12       "role": "system",
13       "content": "WORKING_MEMORY_SYSTEM_INSTRUCTION:\nStore and update any conversation-relevant information by calling the updateWorkingMemory tool. If

```

A state signal underneath. The list lives in the tasks storage domain and is projected onto the state lane, so it survives compaction without touching the cached prompt.

A reactive signal, every step_

transient: delivered for one call, never stored.

```
class SteeringReminderProcessor implements Processor {  
  readonly id = 'steering-reminder'  
  
  async processInputStep({ sendSignal }) {  
    await sendSignal?.({  
      type: 'reactive',  
      transient: true,  
      contents: 'Stay on the current task and keep answers '  
        + 'under three sentences.',  
    })  
  }  
}
```

What transient changes

Without, after three turns

reminder

reminder

reminder

Copies pile up.

With, after three turns

...

reminder

One fresh copy at the tail, never persisted.

Three things to take home_

1

Start from the primitives.

Agent, Workspace, Controller, Memory, or just `createCodingAgent`: most of a coding agent is already there.

2

The real work is the context.

What goes into the prompt, in which order, and what stays out.

3

Look at what the model sees.

Studio and the traces show the request that actually leaves. Most problems only show up there.

All of this applies beyond coding agents. Only `createCodingAgent` is coding-specific.

Thank you. Questions?

mastra.ai/docs

github.com/mastra-ai/mastra

